



HACETTEPE UNIVERSITY
ARTIFICIAL INTELLIGENCE ENGINEERING DEPARTMENT

AIN453 - INTRODUCTION TO ROBOTICS LABORATORY

Final Project Report

June 10, 2026

Student:

Hayrun Nisa Çiko

Zeynep Umay İyim

Mohamed Yahya MANSOURI

Student Number:

2220356043

2220765025

2220765064

Contents

1	Introduction:	2
2	Simulation Environment	2
2.1	Room and World Setup	2
2.2	ArUco Marker Layout	3
2.3	Initial Convergence Behavior	4
2.4	Robot and Camera	4
3	Particle Filter as Bayesian Filtering	5
3.1	The Bayesian Filtering Framework	5
3.2	Particle Representation	5
3.3	Correspondence Between Filter Steps and Bayesian Quantities	6
4	Motion Model (Prediction Step)	6
5	Sensor Model (Update Step)	7
6	Resampling	7
7	Number of Particles and Accuracy	8
8	Visualization	9
8.1	Colour-Coded Particle Cloud	9
8.2	Additional Visual Elements	9
9	System Architecture	10
10	Conclusion	11

1 Introduction:

Localizing a mobile robot without prior knowledge of its starting position is one of the classical problems of autonomous robotics, often called the *global localization* or *kidnapped-robot problem*. In this project we implement a full **particle filter** (also known as a Sequential Monte Carlo filter or Monte Carlo Localization) to estimate the pose (x, y, θ) of a differential-drive robot moving inside a Gazebo simulation.

The environment includes eight ArUco markers mounted on the four walls. Following the project specifications, all markers carry the same ID. As a result, a single camera observation is fundamentally ambiguous: the filter cannot know which physical tag it is looking at. The sensor model must therefore maintain all eight hypotheses simultaneously, a requirement that makes the problem significantly harder than the standard single-landmark case and that is central to the Bayesian treatment described in this report.

In fact, while unique ID markers make the challenge easier, it does not eliminate the necessity of particle filtering as odometry and measurements are noisy, especially in real robots' measurements. It can be argued that this can be mitigated by knowing the location of the ID marker as it gives a strong constraint. Yet, this does not eliminate uncertainty, which is addressed by particle filtering.

The robot is teleoperated by a human operator; the filter consumes odometry velocities and marker range/bearing measurements and continuously updates a cloud of weighted pose hypotheses. As the robot explores the room and accumulates evidence, the cloud progressively narrows to the true pose. The entire simulation runs in real time under ROS 2 Humble and is visualized in RViz2.

The code is shared on github. The duckiebot implementation is shared under a different branch called "duckie-bot."

Source code: <https://github.com/nisaciko/RoboticLab-FinalProject>

- Simulation Branch Name: *main*
- Real Duckiebot Branch Name: *duckie-bot*

Driver Link for videos:

<https://drive.google.com/drive/folders/1RqlCyc240NWvheIyxsPXXxCEwM1I4cah?usp=sharing>

2 Simulation Environment

2.1 Room and World Setup

The simulation is built with **Gazebo Harmonic** (gz-sim 8) and defined in `simulation/worlds/room.sdf`. The environment is a closed rectangular room with interior dimensions of 5 m $\times$ 4 m (width $\times$ depth), centred at the world origin. In the filter's coordinate frame the free space spans $x \in [-2.5, 2.5]$ m and $y \in [-2.0, 2.0]$ m. The walls are 1.6 m tall, 0.1 m thick, and rendered in a neutral grey so that lighting variations do not confuse the ArUco detector.

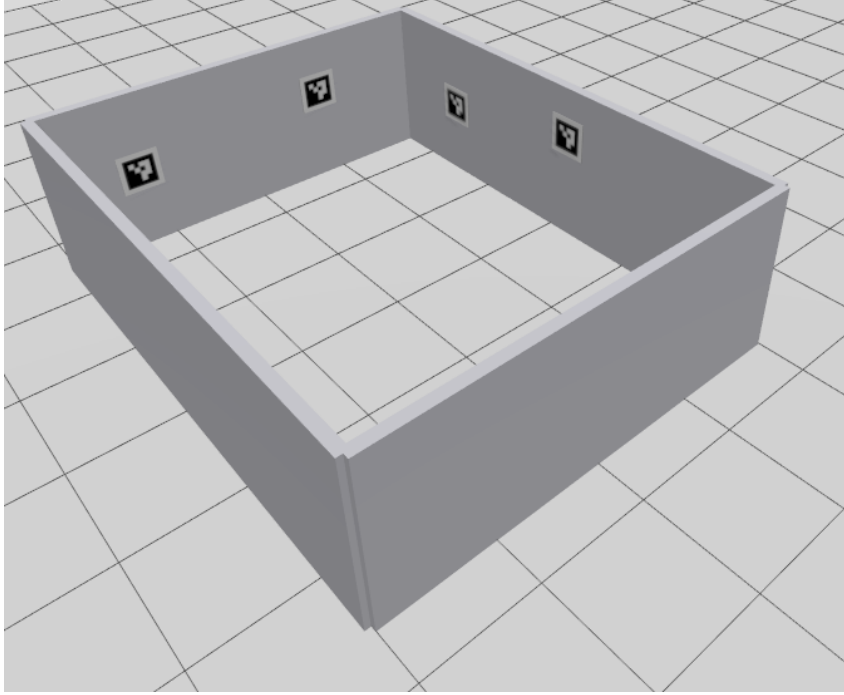


Figure 1: Gazebo simulation environment: A rectangular room with eight ArUco markers, two on each wall.

2.2 ArUco Marker Layout

Eight ArUco markers, with the same ID, are fixed on the four walls at the positions listed in Table 1. Each marker is a $0.4\text{ m} \times 0.4\text{ m}$ square plate (Gazebo box with negligible depth), textured with a PBR albedo map generated by `generate_marker.py`.

Tag #	Wall	x (m)	y (m)
1	Left	-2.44	-1.20
2	Left	-2.44	0.80
3	Right	2.44	0.30
4	Right	2.44	-1.50
5	Front	-1.60	-1.94
6	Front	0.80	-1.94
7	Back	0.20	1.94
8	Back	-1.50	1.94

Table 1: ArUco marker world-frame positions (must match `pf_core/tag_map.py`).

Why asymmetry matters. A perfectly symmetric tag layout, such as two tags equidistant from each corner, would create rotational ambiguities that the filter could never resolve. If the room had 180° -rotational symmetry, an observation from one side of the room would be indistinguishable from an observation made at the mirror-image pose on the other side. The particle cloud would maintain two equally weighted clusters indefinitely, and convergence to a single pose would be impossible without additional information.

The asymmetric placement in Table 1 was designed so that the minimum distance between any tag and its nearest rotationally symmetric counterpart is at least 0.7 m. No corner has two tags within 1 m of each other, which prevents a related “corner-trap” false convergence. As the robot explores the room and accumulates observations from different positions, the wrong clusters are progressively eliminated until only the correct hypothesis survives. The key insight is that *asymmetry is not an aesthetic choice but a mathematical requirement for filter convergence*.

Marker size and detection range. The 0.4 m marker size was chosen deliberately to ensure reliable detection across the full 5 m $\times$ 4 m extent of the room. A marker that is too small (e.g. 0.1 m) would only be detectable at close range and would provide observations too infrequently for the filter to converge in a reasonable time. Conversely, a marker so large that it overflows the camera frame at close range cannot have its corners localised accurately by the ArUco detector. The 0.4 m size occupies a significant fraction of the 640 $\times$ 480 frame even at the far end of the room (roughly 4 m maximum line-of-sight), while staying well within the frame at close range. The same logic applies to the room dimensions: a very large room would require either much larger markers or a higher-resolution camera, while a very small room would compress the particle cloud before a useful trajectory has been driven. Taking these factors into account, different room sizes and marker sizes has been meticulously tested to ensure particle convergence.

2.3 Initial Convergence Behavior

In addition, a spawning position where the markers are not seen by the robot’s camera was favored over the center of the room. The latter can be a starting position as well but further conditions should be added to the simulation to make sure the particles converge when the robot starts moving not when the moment the simulation starts.¹ The problem is that after this premature partial convergence, the subsequent odometry steps must disagree with some of the surviving clusters before those wrong clusters are eliminated. This can require many seconds of motion, during which the filter’s estimate may oscillate or jump. In the worst case, if the initial high-weight cluster happens to be close to a wrong pose, the filter may converge to the wrong location entirely.

2.4 Robot and Camera

The differential-drive robot is defined in `simulation/models/robot/model.sdf`. Its main body (0.4 m $\times$ 0.3 m $\times$ 0.15 m, mass 5 kg) is mounted on two driven wheels (radius 0.1 m, separation 0.4 m) and stabilised by two low-friction casters. An orange nose bump marks the forward (+X) face for visual orientation.

Table 2: Camera intrinsic parameters (from `model.sdf`).

Parameter	Value
Image resolution	640 $\times$ 480 px
Horizontal FOV	2.0 rad ($\approx 114.6^\circ$)
Update rate	30 Hz

¹This is important to demonstrate how the particles converge over time.

The camera has offset of 0.2m forward along the body, facing the $+X$ direction. The Gazebo sensor plugin streams raw images on the `/camera` topic, bridged to ROS 2 by a Python bridge node.

The robot spawns at world coordinates (2.0, -1.5)m facing $-Y$ (yaw = $-\pi/2$), placing it in the front right area of the room with no markers initially in view.

The Gazebo differential-drive plugin subscribes to `/cmd_vel` and publishes odometry on `/odom` at 30 Hz. The robot is teleoperated using arrow keys via a keyboard teleop node launched in a dedicated `xterm` window.

3 Particle Filter as Bayesian Filtering

The particle filter is a non-parametric implementation of the recursive Bayesian filter. To understand why each algorithmic step exists, it is helpful to trace it back to the underlying probabilistic quantities.

3.1 The Bayesian Filtering Framework

We model the robot’s pose at time t as a hidden state $\mathbf{x}_t = (x_t, y_t, \theta_t)^\top$. Given a control input $\mathbf{u}_t$ (the odometry velocity command) and an observation $\mathbf{z}_t$ (range-bearing measurements of visible tags), the recursive Bayesian filter maintains the **belief**

$$\text{bel}(\mathbf{x}_t) = p(\mathbf{x}_t \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}), \quad (1)$$

the posterior probability distribution over all poses consistent with the entire history of measurements and control inputs. The recursion has two steps:

Prediction (prior propagation).

$$\overline{\text{bel}}(\mathbf{x}_t) = \int p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t) \text{bel}(\mathbf{x}_{t-1}) d\mathbf{x}_{t-1}. \quad (2)$$

The *prior* $\overline{\text{bel}}(\mathbf{x}_t)$ is the belief *before* seeing the new observation. It rolls the previous posterior forward through the motion model $p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)$. Because the motion is noisy, the prior is wider (more uncertain) than the previous posterior.

Update (Bayesian correction).

$$\text{bel}(\mathbf{x}_t) = \eta p(\mathbf{z}_t \mid \mathbf{x}_t) \overline{\text{bel}}(\mathbf{x}_t), \quad (3)$$

where $p(\mathbf{z}_t \mid \mathbf{x}_t)$ is the *likelihood* of the observation given a pose hypothesis, and η is a normalisation constant. Hypotheses that predict the observation well receive higher weight; hypotheses that predict it poorly are suppressed.

3.2 Particle Representation

In the particle filter, the continuous belief $\text{bel}(\mathbf{x}_t)$ is approximated by a finite set of N weighted samples (“particles”):

$$\text{bel}(\mathbf{x}_t) \approx \left\{ (\mathbf{x}_t^{[i]}, w_t^{[i]}) \mid i = 1, \dots, N \right\}, \quad \sum_{i=1}^N w_t^{[i]} = 1. \quad (4)$$

Each particle $\mathbf{x}_t^{[i]} = (x^{[i]}, y^{[i]}, \theta^{[i]})$ is a complete pose hypothesis. The weight $w_t^{[i]}$ encodes how well particle i explains the most recent observation. A cloud that is initially spread uniformly over the room represents a *flat prior*—no preference for any pose.

3.3 Correspondence Between Filter Steps and Bayesian Quantities

`initialize_uniform()`

Sets $\text{bel}(\mathbf{x}_0) \propto \text{const}$ — a uniform prior over the free space, expressing total initial ignorance of the robot’s pose.

`predict(u, dt)`

Implements Eq. (2) by sampling from the motion model: $\mathbf{x}_t^{[i]} \sim p(\cdot \mid \mathbf{x}_{t-1}^{[i]}, \mathbf{u}_t)$. This gives the prior $\overline{\text{bel}}(\mathbf{x}_t)$ as a particle approximation.

`update(observations)`

Implements Eq. (3): $w_t^{[i]} \leftarrow w_{t-1}^{[i]} \cdot p(\mathbf{z}_t \mid \mathbf{x}_t^{[i]})$, then normalises. The likelihood $p(\mathbf{z}_t \mid \mathbf{x}_t^{[i]})$ is the multi-hypothesis sensor model.

`resample()`

Replaces the particle set with N samples drawn *with replacement* in proportion to the weights. This concentrates particles in high-posterior regions and discards low-likelihood hypotheses, keeping the posterior representation numerically healthy.

`estimate()`

Computes the **MMSE estimate**: the weighted mean $\hat{\mathbf{x}}_t = \sum_i w_t^{[i]} \mathbf{x}_t^{[i]}$ (with circular mean for θ). This is the point-estimate of the posterior.

4 Motion Model (Prediction Step)

The robot is modelled as a differential-drive vehicle with state $\mathbf{x} = (x, y, \theta)^\top$. At each odometry callback the control input $u_t = [v_t, \omega_t]^\top$ (linear velocity in m s^{-1} , angular velocity in rad s^{-1}) and timestep Δt are used to propagate every particle via:

$$\hat{v}^{[i]} = v_t + \varepsilon_v^{[i]}, \quad \varepsilon_v^{[i]} \sim \mathcal{N}(0, \sigma_v^2), \quad (5)$$

$$\hat{\omega}^{[i]} = \omega_t + \varepsilon_\omega^{[i]}, \quad \varepsilon_\omega^{[i]} \sim \mathcal{N}(0, \sigma_\omega^2), \quad (6)$$

$$\begin{pmatrix} x' \\ y' \\ \theta' \end{pmatrix}^{[i]} = \begin{pmatrix} x^{[i]} + \hat{v}^{[i]} \cos \theta^{[i]} \Delta t \\ y^{[i]} + \hat{v}^{[i]} \sin \theta^{[i]} \Delta t \\ \text{wrap}(\theta^{[i]} + \hat{\omega}^{[i]} \Delta t) \end{pmatrix}, \quad (7)$$

where $\text{wrap}(\cdot) = \text{atan2}(\sin(\cdot), \cos(\cdot))$ keeps the heading in $(-\pi, \pi]$. Independent Gaussian noise is added to each particle’s velocity sample so that the cloud spreads to reflect the uncertainty in odometry. The noise standard deviations used in the live simulation are

$$\sigma_v = 0.10 \text{ m s}^{-1}, \quad \sigma_\omega = 0.05 \text{ rad s}^{-1}. \quad (8)$$

After propagation, particles that land outside the room boundaries are clamped to the nearest free-space coordinate, effectively implementing a hard wall constraint.

5 Sensor Model (Update Step)

The sensor model answers the question: how likely is it that a robot at pose $\mathbf{x}^{[i]}$ would produce the observation $z = (r, \beta)$ that was just received? The answer is used to re-weight every particle. Particles at poses that are geometrically consistent with the measurement receive higher weights, while inconsistent ones are down-weighted and eventually die out.

Range and bearing. Each camera detection is converted to a (range, bearing) pair (r, β) by the `apriltag_obs_node`. For a particle at pose $(x^{[i]}, y^{[i]}, \theta^{[i]})$ and a tag at known world position (t_x^j, t_y^j) , the predicted range and bearing are:

$$\hat{r}_{ij} = \sqrt{(t_x^j - x^{[i]})^2 + (t_y^j - y^{[i]})^2}, \quad (9)$$

$$\hat{\beta}_{ij} = \text{atan2}(t_y^j - y^{[i]}, t_x^j - x^{[i]}) - \theta^{[i]}. \quad (10)$$

Multi-hypothesis likelihood. Because all eight tags share the same ID, it is impossible to know which physical tag produced a detection. Rather than guessing or taking the nearest tag, the likelihood sums over all eight tags as equally valid hypotheses:

$$p(z \mid \mathbf{x}^{[i]}) = \sum_{j=1}^8 \exp\left(-\frac{(r - \hat{r}_{ij})^2}{2\sigma_r^2} - \frac{(\beta - \hat{\beta}_{ij})^2}{2\sigma_\beta^2}\right) \cdot \mathbf{1}\left[|\hat{\beta}_{ij}| \leq \Delta_{\text{FOV}}\right]. \quad (11)$$

The indicator term discards any tag that lies outside the camera’s field of view, since a physically invisible tag cannot be the source of a detection. When a frame contains multiple detections their likelihoods are multiplied together (conditional independence assumption) and the result is used to scale the particle’s weight. All weights are then normalised to sum to one.

Parameter	Symbol	Value
Range noise std dev	σ_r	0.30 m
Bearing noise std dev	σ_β	0.18 rad
Camera half-FOV	Δ_{FOV}	1.0 rad ($\approx 57.3^\circ$)

Table 3: Sensor model noise parameters.

6 Resampling

After the weight update, the distribution is represented by particles with highly unequal weights: a few particles near the true pose carry most of the probability mass, while the majority contribute negligibly. Without resampling, subsequent prediction steps would waste computation propagating these low-weight particles. Resampling replaces the weighted set with N unweighted particles drawn proportionally to the weights.

Resampling is applied only when the particle weights have become unbalanced enough to matter. This is measured by the effective sample size:

$$N_{\text{eff}} = \frac{1}{\sum_{i=1}^N (w^{[i]})^2}. \quad (12)$$

When all weights are equal $N_{\text{eff}} = N$; when one particle holds all the weight $N_{\text{eff}} = 1$. Resampling is triggered whenever $N_{\text{eff}} < N/2 = 4,000$, i.e. when the distribution has effectively collapsed to fewer than half the intended particles.

Low-variance (systematic) resampling is used instead of drawing N independent random numbers. A single random offset generates all selection thresholds, which gives a more representative sample and reduces noise.

After resampling, many particles are exact duplicates of the high-weight originals. If left as-is, the cloud would collapse to a handful of distinct points and the filter would lose its ability to recover from a wrong hypothesis. To prevent this, a small random jitter (roughening) is added to each particle:

$$x^{[i]} += \mathcal{N}(0, 0.10^2), \quad y^{[i]} += \mathcal{N}(0, 0.10^2), \quad \theta^{[i]} += \mathcal{N}(0, 0.05^2). \quad (13)$$

This keeps the cloud spread out enough to explore nearby poses without undoing the convergence that the update step achieved.

7 Number of Particles and Accuracy

The choice of particle count N involves a direct trade-off between computational cost and filter accuracy. We use $N = 8,000$ particles in the live simulation. The rationale is as follows:

- With $N = 8,000$ particles distributed across the $5 \times 4 \text{ m}^2 = 20 \text{ m}^2$ room interior, the average initial density is 400 particles/ m^2 . This is sufficient to have at least a few particles within $\pm 5 \text{ cm}$ of the true pose at initialisation, ensuring that the first meaningful observation does not reduce every particle’s weight to near zero.
- Too few particles (e.g. $N = 100$) create a sparse representation that misses low-probability regions. The filter may “lose” the robot entirely after an aggressive turn or an ambiguous observation because no particle is close enough to the true pose to receive a significant weight.
- Too many particles (e.g. $N = 50,000$) increase per-cycle compute time proportionally. At 30 Hz odometry the filter must complete a full predict–update–resample cycle within $\approx 33 \text{ ms}$. With $N = 8,000$ particles, the vectorised NumPy operations in our implementation comfortably meet this budget on a standard laptop CPU.
- In the offline convergence test (`tests/test_offline.py`), $N = 2,000$ is sufficient to reach a final position error below 0.5 m after 200 steps. The live simulation uses $N = 8,000$ for a smoother cloud appearance and faster convergence from the initial uniform prior.

In general, the required N grows with the volume of the state space and the sharpness of the likelihood function. For our 3-DOF pose space and moderately noisy Gaussian likelihoods, $N = 8,000$ provides a practical balance.

8 Visualization

Real-time visualisation is provided by the `viz_node`, which publishes a `MarkerArray` on the `/pf/viz` topic at 10 Hz (dynamic markers) and 1 Hz (static markers). Open RViz2 with Fixed Frame set to `odom` and add the `MarkerArray` display.

8.1 Colour-Coded Particle Cloud

Each particle is rendered as a small point (0.04 m radius) with colour interpolated linearly between **red** (weight = 0) and **green** (weight = $w_{\max}$):

$$\text{colour}^{[i]} = (1 - t^{[i]}, t^{[i]}, 0, 0.7)_{\text{RGBA}}, \quad t^{[i]} = \frac{w^{[i]}}{w_{\max}}. \quad (14)$$

This encoding gives an immediate visual read of the filter’s certainty at a glance:

- **Red particles:** low-weight hypotheses, inconsistent with recent observations. A cloud that is entirely red indicates the filter has not yet received useful measurements.
- **Green particles:** high-weight hypotheses, consistent with recent observations. A tightly packed green cluster indicates the filter has localised the robot.
- **Mixed cloud:** a partially converged state where multiple competing hypotheses still exist—typically occurring when one wall’s two tags have been observed but the robot has not yet moved far enough to distinguish their relative arrangement.

This red-to-green gradient is far more informative than a single-colour cloud because it immediately reveals not just where the particles are but how much the filter trusts each location. An operator can tell at a glance whether the cloud has collapsed around a single consistent hypothesis (all green in one cluster) or whether competing hypotheses are still alive (green blobs at multiple locations).

8.2 Additional Visual Elements

- **Room outline:** a grey `LINE_STRIP` tracing the 5×4 m boundary.
- **Tag cubes:** orange `CUBE` markers (0.4 m face) at each of the eight known tag positions.
- **Odometry trajectory** (blue): the path the robot *would* follow if it trusted odometry alone, without any sensor corrections. This trail drifts away from the true path over time, illustrating dead-reckoning error accumulation.
- **PF estimate trajectory** (green): the weighted-mean pose estimate over time. After convergence, this trail closely matches the robot’s true world-frame path, demonstrating that the filter has corrected the odometry drift.

- **Ground-truth sphere** (cyan): the robot's true world pose from Gazebo's PosePublisher plugin, used to verify that the green particle cluster is indeed co-located with the robot.

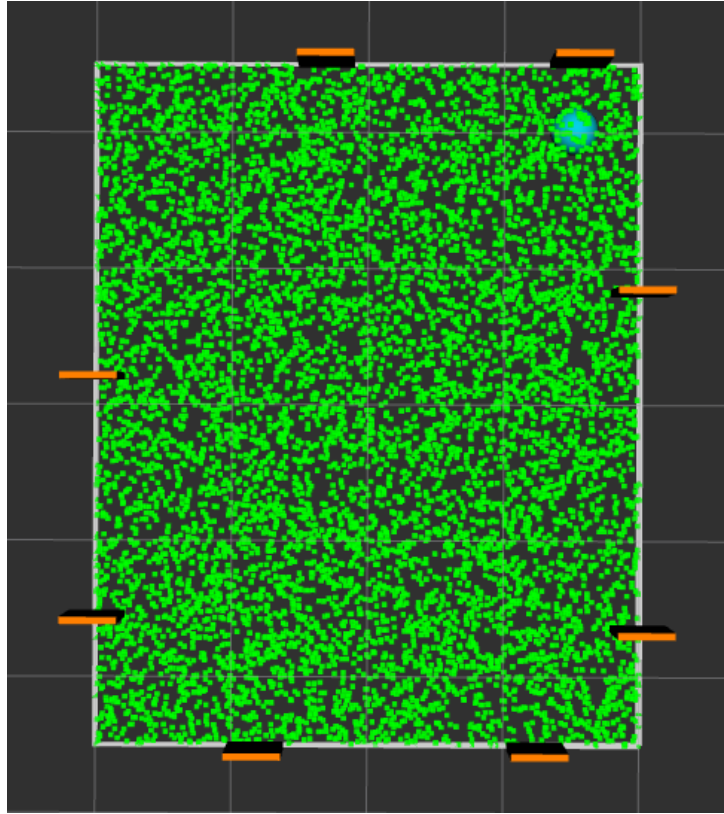


Figure 2: RViz2 Visualization of Initial Particle Cloud

9 System Architecture

The code is organized into two packages: a ROS-free Python library (`pf_core/`) containing all algorithmic components, and a ROS 2 package (`pf_ros2/`) containing the adapter nodes. This separation allows the core filter to be unit-tested without a ROS installation.

`pf_core/particle_filter.py` End-to-end filter: `initialize_uniform`, `predict`, `update`, `resample`, `step`, `estimate`.

`pf_core/motion_model.py` `sample_motion`: vectorised prediction over all N particles using NumPy broadcasting.

`pf_core/sensor_model.py` `observation_likelihood`: $(N \times T)$ matrix computation of the multi-hypothesis likelihood.

`pf_core/resampling.py` `low_variance_resample`, `roughen`, `effective_sample_size`.

`pf_core/tag_map.py` `TagMap`: world-frame tag positions, room bounds, asymmetric 8-tag default layout.

`pf_ros2/pf_node.py` Bridges odometry and observations to `pf_core.ParticleFilter`; publishes pose, cloud, and weights.

`pf_ros2/apriltag_obs_node.py` Decodes camera frames, runs the ArUco detector, converts detections to (r, β) pairs.

`pf_ros2/viz_node.py` Produces `MarkerArray` for RViz2.

10 Conclusion

In this project, we implemented a particle filter to solve the global localization problem for a differential-drive robot in a simulated room. The robot has no knowledge of its starting position, and all eight ArUco markers share the same ID, which makes the problem genuinely ambiguous. The filter handles this by keeping all eight tags as possible sources of every detection and summing their likelihoods rather than committing to one.

Several practical choices turned out to matter more than expected. The asymmetric marker placement was necessary, not just convenient because a symmetric layout would have left the filter permanently split between two mirror-image hypotheses. The marker size and particle count both required tuning: too small or too few and the filter fails to converge in reasonable time; too large or too many and detection or real-time performance breaks down. Spawning the robot without any markers in view also made a noticeable difference, as it prevents the filter from partially converging before the robot has actually moved.

The result is a system that starts with 8,000 particles spread uniformly across the room and, as the robot drives and accumulates observations, collapses to a tight cluster around the true pose. The RViz2 visualization makes this process easy to follow and verify against the Gazebo ground truth.

Source code: <https://github.com/nisaciko/RoboticLab-FinalProject>

- Simulation Branch Name: *main*
- Real Duckiebot Branch Name: *duckie-bot*

Driver Link for videos:

<https://drive.google.com/drive/folders/1Rq1Cyc240NWvheIyxsPXXxCEwM1I4cah?usp=sharing>